

Genetic Algorithms

A Simple Introduction

An easy, example-based guide for undergraduate students

Dr. Kanchan Rajwar

Department of Mathematics

Indian Institute of Information Technology, Bhopal, India

Email: kanchan.rajwar@iiitbhopal.ac.in

Optimization Techniques • CSE(DS)-5003 • Module 3

To the Student

This small book explains Genetic Algorithms (GA) in simple words. It is written for students who are meeting the topic for the first time. Every idea is built slowly — first we see why the idea is needed, then we learn the idea, then we work through an example. You do not need advanced mathematics. You only need basic algebra and a little binary counting.

Read the chapters in order. Try the examples with pen and paper as you go. By the end, you will be able to run a Genetic Algorithm by hand and answer exam questions with confidence.

How to use these notes:

1. Attend the class lecture for each topic.
2. Read the corresponding section of these notes carefully, working through each example yourself.
3. Once you have finished a chapter, attempt the related questions in the companion question bank (*GA Module 3 — Question Bank*) for self-assessment.
4. Check your answers against the answer key only *after* attempting the questions on your own.

The question bank contains 100 multiple choice questions, 5 short answer questions, and 5 broad questions — all based directly on these notes and the running example developed in class. Regular self-assessment is one of the most effective ways to prepare for exams. Do not skip it.

Good luck, and enjoy the topic.

Contents

Preface	4
1 Introduction	5
1.1 Why Classical Methods Are Not Always Sufficient	5
1.2 Learning from Nature	6
1.3 What is a Genetic Algorithm?	6
2 Basic Terms	7
2.1 Fitness	8
3 How a GA Works	9
3.1 Encoding the Solution	10
3.1.1 Binary Encoding	10
3.1.2 Decoding to a Range	10
3.1.3 Precision	11
3.1.4 Other Encodings	11
3.2 The Fitness Function	11
3.2.1 Maximisation Problems	11
3.2.2 Minimisation Problems	11
3.3 Selection	12
3.3.1 Roulette-Wheel Selection	12
3.4 Crossover	14
3.4.1 Single-Point Crossover	14
3.4.2 Other Crossover Operators	15
3.5 Mutation	15
3.5.1 Bit-Flip Mutation	15
3.5.2 Other Mutation Operators	16
3.6 A Complete GA Run	17
3.6.1 Elitism	17
3.7 Parameters and Practical Guidelines	17
3.7.1 The Main Parameters	17
3.7.2 Premature Convergence	18
4 Advantages and Limitations	19
4.1 Advantages	19
4.2 Limitations	19

5 Conclusion	20
Python Implementation	21

Preface

This book is a module-level introduction to Genetic Algorithms, prepared for students of Optimization Techniques (CSE(DS)-5003) at the Indian Institute of Information Technology, Bhopal. It forms Module 3 of the course.

The notes are written to be self-contained. A student who reads carefully, works through every example, and attempts the companion question bank should be fully prepared for examination questions on this topic.

The running example — maximising $f(x) = x^2$ over $x \in [0, 10]$ using 4-bit binary chromosomes and a population of four — appears in every chapter. This consistency is deliberate: it allows the reader to see exactly how each operator acts on the same data, making it easier to understand the algorithm as a whole rather than as a collection of isolated steps.

Comments and suggestions for improvement are welcome.

Dr. Kanchan Rajwar

Department of Mathematics

Indian Institute of Information Technology, Bhopal, India

kanchan.rajwar@iiitbhopal.ac.in

August 2026

1. Introduction

Optimization is the process of finding the best possible solution to a problem under a given set of conditions. Depending on the context, “best” may mean maximizing profit, minimizing cost, finding the shortest path, or achieving the highest accuracy — the objective varies, but the goal is always the same: identify the value of x that makes $f(x)$ as good as it can be.

Formally, given an objective function $f(x)$, optimization seeks

$$x^* = \arg \max_{x \in \mathcal{X}} f(x) \quad \text{or} \quad x^* = \arg \min_{x \in \mathcal{X}} f(x),$$

depending on whether the problem calls for maximization or minimization. Throughout this article, “best” will consistently refer to the largest fitness value, i.e., the maximization setting.

1.1 Why Classical Methods Are Not Always Sufficient

Classical optimization methods — such as gradient descent and Newton’s method — are highly effective when the objective function is smooth, continuous, and differentiable. By following the slope of the function, these methods efficiently locate a local optimum. However, many real-world problems do not satisfy these conditions.

- **Discrete problems:** In many practical settings, the decision variable is not a continuous number but a choice, a sequence, or a permutation. Finding the optimal route through a set of cities, for instance, involves selecting an ordering — a structure that has no meaningful slope to follow.
- **Multimodal problems:** Many objective functions contain multiple local optima — numerous hills and valleys spread across the search space. Gradient-based methods tend to converge to whichever local optimum is nearest to the starting point, with no guarantee of reaching the global optimum.
- **Black-box problems:** In some applications, no closed-form expression for the objective function is available. Only input-output evaluations are accessible — a simulation, an experiment, or an external process — making gradient computation impossible.
- **NP-hard problems:** A particularly important class of problems are those classified as NP-hard, for which no known algorithm can guarantee an optimal solution in polynomial time. As the problem size grows, the number of possible solutions increases combinatorially, making exhaustive search computationally infeasible. Classic examples include the Travelling Salesman Problem (TSP), the Knapsack Problem, and graph coloring. For such problems, exact methods either fail to scale or require prohibitive computational resources.

In each of these settings, classical methods are either inapplicable or unreliable. This motivates the need for a fundamentally different class of optimization techniques — ones capable of searching large, complex spaces efficiently without relying on smoothness, gradients, or exhaustive enumeration. The following sections introduce one such family of methods.

1.2 Learning from Nature

Nature solves a hard problem every day: design organisms that survive in a complex, changing world. There is no formula and no gradient — yet over millions of years, life keeps improving.

The rule is simple. Individuals better suited to their environment survive and reproduce. Their offspring inherit parental traits, but with small random variations. Good variations persist; harmful ones disappear. Repeated across generations, this pushes the population toward better and better solutions.

Darwin called this *natural selection*. It is powerful because it needs no knowledge of the problem's structure — only a way to measure how well a candidate performs.

A Genetic Algorithm (GA) borrows this idea. Instead of organisms, it evolves a population of candidate solutions. Instead of survival, it uses an objective function to measure fitness. Instead of biological reproduction, it applies crossover and mutation to generate new solutions — repeating this process generation after generation until a good answer is found.

1.3 What is a Genetic Algorithm?

A **Genetic Algorithm (GA)** is a population-based metaheuristic inspired by Darwinian evolution. It maintains a collection of candidate solutions — called a **population** — and improves them over successive generations using three core operators:

- **Selection:** better solutions are chosen more often to pass their traits forward.
- **Crossover:** two parent solutions are combined to produce new offspring solutions.
- **Mutation:** small random changes are introduced to maintain diversity and explore new regions of the search space.

The process begins with a randomly initialized population and repeats — selecting, recombining, and mutating — until a satisfactory solution is found or a stopping criterion is met.

The key advantage of a GA is that it requires no gradient and no analytical formula. It needs only a way to score each candidate solution. This makes it naturally suited to the problem classes discussed in the previous section: discrete, multimodal, black-box, and NP-hard problems alike.

Summary

- Optimization means finding the best answer.
- Old methods need a smooth graph and a slope.
- Many real problems are discrete, bumpy, or black-box.
- GA copies natural selection and needs only fitness, not a slope.

2. Basic Terms

Genetic Algorithms borrow their vocabulary from biology, but each term has a precise and simple meaning in this context. This chapter introduces the key definitions needed to follow the rest of the article.

- **Chromosome:** A **chromosome** (also called a string or individual) is a single candidate solution in encoded form. For example, suppose we are scheduling four tasks and encoding the order as a binary string — the chromosome 1011 represents one particular ordering of those tasks.

Every chromosome has two sides. The **genotype** is the encoded string that the GA works with directly. The **phenotype** is the actual solution this string represents in the problem domain. *Encoding* maps a phenotype to its genotype; *decoding* reverses the process. The GA always operates on genotypes, but fitness is always evaluated on the decoded phenotype.

The decoding process depends on the nature of the problem.

Case 1: Integer problems. The phenotype is simply the decimal equivalent of the binary genotype.

$$11010 \rightarrow 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 26.$$

Case 2: Continuous problems. When the variable x takes real values in a bounded interval $[x_{\min}, x_{\max}]$, the binary string first decodes to an integer, which is then mapped to the target range using

$$x = x_{\min} + \frac{x_{\max} - x_{\min}}{2^L - 1} \times (\text{integer value}),$$

where L is the number of bits in the chromosome. This scaling step is necessary because a raw binary integer carries no information about the range or units of the actual variable.

Example 2.1. Let $L = 5$, $x \in [0, 10]$, and consider two chromosomes.

Genotype	Integer	Phenotype x
11010	26	$\frac{10}{31} \times 26 \approx 8.39$
00110	6	$\frac{10}{31} \times 6 \approx 1.94$

The GA always operates on the genotype, but the real-world meaning of each chromosome is its decoded phenotype.

- **Gene:** A **gene** is the smallest unit of a chromosome — one component of the encoded solution. In a binary chromosome, each gene is a single bit. In the chromosome 1011, there are four genes: 1, 0, 1, and 1. Each gene contributes one piece of information to the overall solution.
- **Population:** A **population** is the complete set of chromosomes maintained at any point during the algorithm. For example, a population of four chromosomes might look like:

$$\{ 1011, 0110, 1100, 0001 \}.$$

Each string is a different candidate solution. Unlike classical methods (e.g. Newton–Raphson) that refine a single solution, a GA evolves the entire population simultaneously, exploring many regions of the search space at once.

2.1 Fitness

The **fitness** of a chromosome is a scalar value that measures how good that candidate solution is with respect to the problem objective. It is the GA’s only way of judging quality — there is no gradient, no formula to differentiate, just a score.

A higher fitness indicates a better solution. The GA uses this score to guide the search: chromosomes with higher fitness are more likely to be selected for reproduction, passing their traits to the next generation, while weaker chromosomes are gradually phased out.

Example 2.2. Suppose the problem is to maximize $f(x) = x^2$ over $x \in [0, 10]$, using 4-bit binary chromosomes ($L = 4$, so $2^L - 1 = 15$). Each chromosome is first decoded to an integer, then mapped to a real value using

$$x = 0 + \frac{10}{15} \times (\text{integer value}).$$

The table below shows the full pipeline from genotype to fitness.

Genotype	Integer	Phenotype x	Fitness $F(x) = x^2$
1011	11	7.33	53.73
0110	6	4.00	16.00
1100	12	8.00	64.00
0001	1	0.67	0.44

Here, 1100 has the highest fitness and is therefore the most likely to be selected. 0001 has the lowest fitness and is likely to be discarded.

For a minimization problem, where a smaller $f(x)$ is better, the fitness function is typically transformed as

$$F(x) = \frac{1}{1 + f(x)},$$

so that a lower objective value maps to a higher fitness score, keeping the GA’s selection mechanism consistent.

3. How a GA Works

A GA repeats the same six steps until a stopping criterion is met.

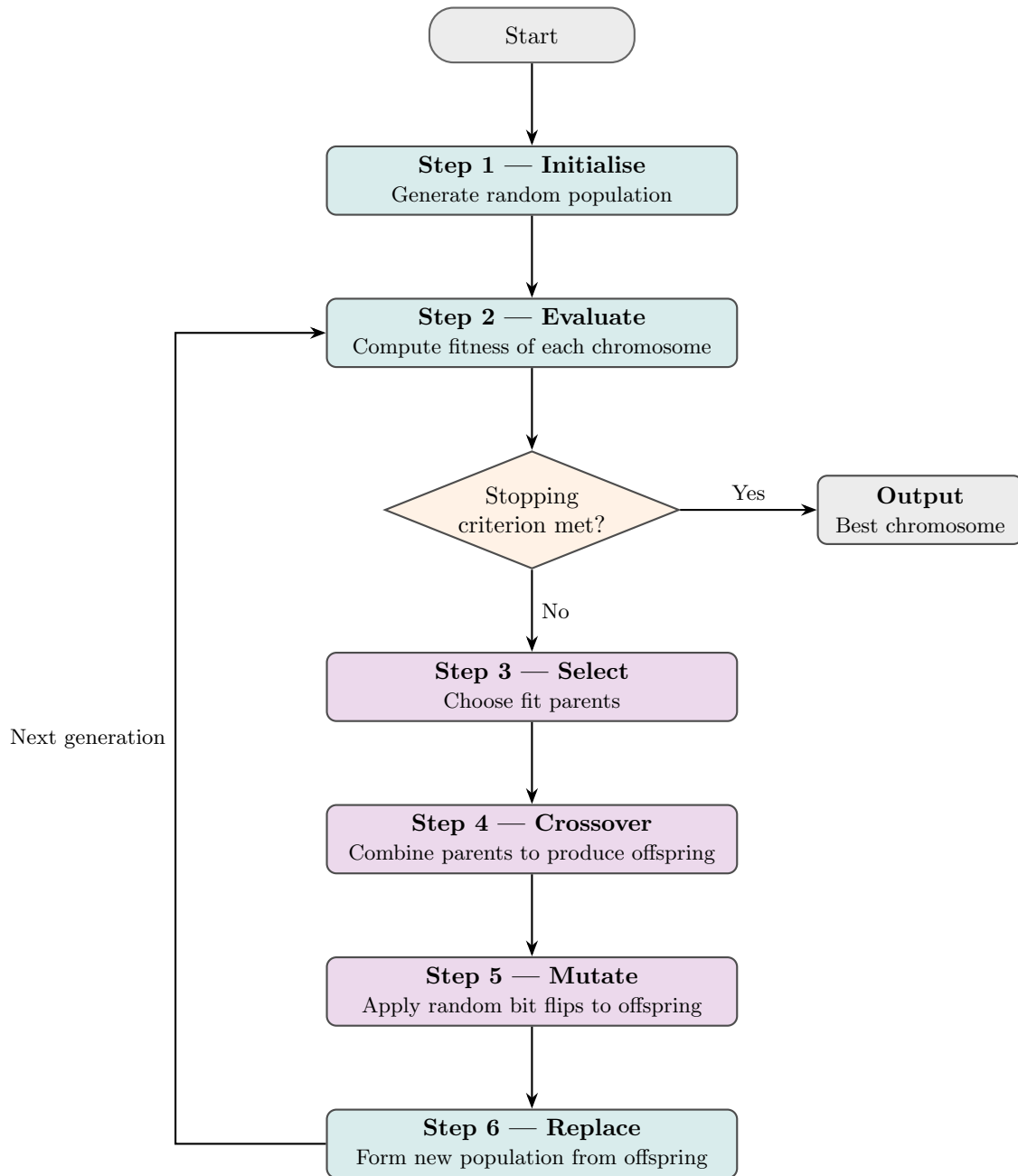


Figure 3.1: Flowchart of the Genetic Algorithm main loop.

Steps 3, 4, and 5 are the three **genetic operators** — the engine of the GA. Together, they balance exploration of new regions of the search space with exploitation of already-good solutions. The loop stops when a fixed number of generations has elapsed, when the best fitness has not improved over many consecutive generations, or when an acceptable solution has been found.

A compact pseudocode summary is given below.

Algorithm 1: Genetic Algorithm

Generate random initial population
 Evaluate fitness of each chromosome
repeat
 Select parents from current population
 Apply crossover to produce offspring
 Apply mutation to offspring
 Evaluate fitness of offspring
 Form new population
until *stopping criterion is met*
 Output the chromosome with the highest fitness

3.1 Encoding the Solution

The genetic operators work by cutting, joining, and flipping parts of a solution. A plain real number cannot be manipulated this way. We therefore represent each candidate solution as a string of symbols — a process called **encoding**. The encoded string is the **genotype**; the decoded real value is the **phenotype**.

3.1.1 Binary Encoding

The most common encoding uses bits (0 and 1). A chromosome of length L bits can represent 2^L distinct values. For example, $L = 4$ bits can represent $2^4 = 16$ integers from 0 to 15.

To decode a binary string to an integer, sum its place values.

Example 3.1. Decode 1011.

$$1011 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11.$$

3.1.2 Decoding to a Range

When x takes continuous values in $[x_{\min}, x_{\max}]$, the decoded integer is further scaled using

$$x = x_{\min} + \frac{x_{\max} - x_{\min}}{2^L - 1} \times (\text{integer value}).$$

The denominator $2^L - 1$ accounts for the fact that 2^L distinct strings produce $2^L - 1$ equally spaced intervals between $x_{\min}$ and $x_{\max}$.

Example 3.2. For $f(x) = x^2$ with $x \in [0, 10]$ and $L = 4$ bits ($2^4 - 1 = 15$), decode the four chromosomes below.

Genotype	Integer	Phenotype x	Fitness $f(x) = x^2$
1011	11	$\frac{10}{15} \times 11 = 7.33$	53.73
0110	6	$\frac{10}{15} \times 6 = 4.00$	16.00
1100	12	$\frac{10}{15} \times 12 = 8.00$	64.00
0001	1	$\frac{10}{15} \times 1 = 0.67$	0.44

These four chromosomes will be used as the running example throughout the rest of this section.

3.1.3 Precision

The **precision** is the smallest difference between two consecutive decoded values:

$$\delta = \frac{x_{\max} - x_{\min}}{2^L - 1}.$$

To achieve a desired precision ε , choose L such that $2^L - 1 \geq (x_{\max} - x_{\min}) / \varepsilon$.

Example 3.3. For $x \in [0, 10]$ with $\varepsilon = 0.01$:

$$2^L - 1 \geq \frac{10}{0.01} = 1000 \implies 2^L \geq 1001.$$

Since $2^9 = 512 < 1001$ and $2^{10} = 1024 \geq 1001$, we need $L = 10$ bits. The achieved precision is $10/1023 \approx 0.0098$.

3.1.4 Other Encodings

- **Real-coded encoding** stores real numbers directly without converting to binary. It is preferred for continuous engineering problems where high precision is required.
- **Permutation encoding** represents a solution as an ordering of elements, used for combinatorial problems such as the Travelling Salesman Problem (TSP). Binary encoding is unsuitable here, since a random binary string may repeat or omit cities, producing an invalid route.

3.2 The Fitness Function

The **objective function** $f(x)$ measures the quantity we care about. The **fitness function** $F(x)$ is the score the GA uses internally to guide selection. The GA always maximises fitness, so $F(x)$ must be designed to reflect the goal of the problem.

3.2.1 Maximisation Problems

When the goal is to maximise $f(x)$, set $F(x) = f(x)$ directly.

Example 3.4. Continuing the running example ($f(x) = x^2$, $x \in [0, 10]$, $L = 4$), the fitness values were already computed during decoding.

Genotype	Phenotype x	Fitness $F(x) = x^2$
1011	7.33	53.73
0110	4.00	16.00
1100	8.00	64.00
0001	0.67	0.44

1100 has the highest fitness and 0001 the lowest.

3.2.2 Minimisation Problems

When the goal is to minimise $f(x)$, a smaller objective value must map to a higher fitness. Two common transformations are

$$F(x) = \frac{1}{1 + f(x)} \quad \text{or} \quad F(x) = C_{\max} - f(x),$$

where $C_{\max}$ is a constant larger than any value $f(x)$ is expected to take. A safe choice is $C_{\max} = \max_i f(x_i) + 1$ over the current population.

Example 3.5. If instead we minimise $f(x) = x^2$, applying $F(x) = 1/(1 + f(x))$ gives:

Genotype	Phenotype x	$f(x)$	$F(x) = \frac{1}{1+f(x)}$
1011	7.33	53.73	0.018
0110	4.00	16.00	0.059
1100	8.00	64.00	0.015
0001	0.67	0.44	0.694

Now 0001 correctly has the highest fitness, identifying $x \approx 0$ as the minimiser.

3.3 Selection

Selection determines which chromosomes become parents. Fitter chromosomes should be chosen more often, but weaker ones should not be excluded entirely — they may carry useful genes. The goal is to apply selection pressure without eliminating diversity.

3.3.1 Roulette-Wheel Selection

Each chromosome receives a slice of a wheel proportional to its fitness. A random spin is more likely to land on a larger slice.

- Compute fitness f_i for each chromosome i .
- Compute selection probability $p_i = f_i / \sum_j f_j$.
- Compute cumulative probability $P_i = \sum_{k=1}^i p_k$.
- Draw $r \sim \text{Uniform}[0, 1]$.
- Select the first chromosome for which $P_i \geq r$.
- Repeat Steps 4–5 until the mating pool is full.

The **expected count** $f_i/\bar{f}$ indicates how many times chromosome i is expected to appear in the mating pool.

Example 3.6. Apply roulette-wheel selection to the running example.

Genotype	$f(x)$	p_i	P_i	Exp. count
1011	53.73	0.401	0.401	1.60
0110	16.00	0.119	0.520	0.48
1100	64.00	0.477	0.997	1.91
0001	0.44	0.003	1.000	0.01
Sum	134.17	1.000		4.00

Drawing four random numbers $r = 0.21, 0.65, 0.38, 0.94$ and comparing each against the cumulative probabilities P_i determines which chromosome is selected each time. The rule is: select the first chromosome whose $P_i \geq r$.

Draw	Comparison	Selected
$r = 0.21$	$P_1 = 0.401 \geq 0.21$	1011
$r = 0.65$	$P_1 = 0.401 < 0.65$, $P_2 = 0.520 < 0.65$, $P_3 = 0.997 \geq 0.65$	1100
$r = 0.38$	$P_1 = 0.401 \geq 0.38$	1011
$r = 0.94$	$P_1 = 0.401 < 0.94$, $P_2 = 0.520 < 0.94$, $P_3 = 0.997 \geq 0.94$	1100

The mating pool is therefore:

$$\{1011, 1100, 1011, 1100\}.$$

1100 is selected twice because it holds the largest slice of the wheel ($p_3 = 0.477$, nearly half the total). Both $r = 0.65$ and $r = 0.94$ fall inside its cumulative interval $(0.520, 0.997]$, so it wins both spins. 1011 is also selected twice since its slice $(0, 0.401]$ captures both $r = 0.21$ and $r = 0.38$. 0110 and 0001 receive no selections: their slices are so small ($p = 0.119$ and $p = 0.003$) that none of the four random draws landed on them.

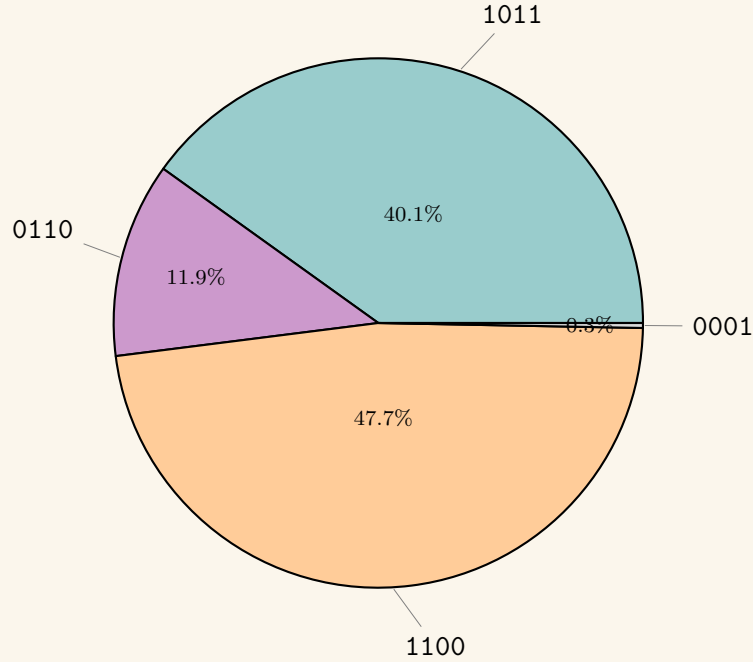


Figure 3.2: Roulette-wheel selection probabilities for $f(x) = x^2$, $x \in [0, 10]$, $L = 4$. Slice size is proportional to fitness. A random draw r selects the chromosome whose slice it lands in: 1100 occupies nearly half the wheel ($p = 0.477$), so it wins any draw falling in $(0.520, 0.997]$. Both $r = 0.65$ and $r = 0.94$ fall in that interval, giving 1100 two selections.

Other Selection Methods

- **Tournament Selection:** A random subset of k chromosomes is drawn. The fittest in the subset wins and becomes a parent. Simple, efficient, and requires no global probability computation.
- **Rank Selection:** Chromosomes are ranked from worst to best. Selection probability is based on rank rather than raw fitness, moderating pressure when fitness values are spread very unevenly.

3.4 Crossover

Two parents may each contain a different good partial solution. Crossover recombines them in the hope that an offspring inherits the good parts of both.

3.4.1 Single-Point Crossover

A single cut point is chosen uniformly at random from positions 1 to $L - 1$, where L is the chromosome length. For a 4-bit chromosome, the cut can fall after position 1, 2, or 3 — each equally likely. The portion of each chromosome after the cut point is then exchanged, producing two offspring.

Example 3.7. Recall the mating pool from the selection step:

$$\{1011, 1100, 1011, 1100\}.$$

We pair them as (1011, 1100) and (1011, 1100) and set $p_c = 0.8$. For each pair, we draw a random number $r \sim \text{Uniform}[0, 1]$. Crossover is performed only if $r \leq p_c$; otherwise the parents are copied unchanged.

Pair 1: $r = 0.43 \leq 0.8$ — crossover is performed. Cut randomly falls after position 2.

$$\begin{array}{ccc} 10|11 & \xrightarrow{\text{swap tails}} & 1000 \\ 11|00 & & 1111 \end{array}$$

Pair 2: $r = 0.91 > 0.8$ — crossover is skipped. Parents are copied unchanged.

$$\begin{array}{ccc} 1011 & \xrightarrow{\text{copied}} & 1011 \\ 1100 & & 1100 \end{array}$$

Decoding all four offspring for $x \in [0, 10]$, $L = 4$ ($2^4 - 1 = 15$):

Offspring	Integer	Phenotype x	Fitness x^2
1000	8	5.33	28.44
1111	15	10.00	100.00
1011	11	7.33	53.73
1100	12	8.00	64.00

The new population after crossover is {1000, 1111, 1011, 1100}, with average fitness

$$\bar{F} = \frac{28.44 + 100.00 + 53.73 + 64.00}{4} = 61.54.$$

This is already an improvement over the initial average of 33.54. The offspring 1111 reaches the global optimum $f = 100$, outperforming both of its parents (53.73 and 64.00). Pair 2, having been copied unchanged, contributes the same chromosomes as before — they remain candidates for improvement in the mutation step.

Note on Crossover Rate: Crossover is applied typically with probability $p_c \in [0.6, 0.9]$. For each pair, a random number $r \sim \text{Uniform}[0, 1]$ is drawn: if $r \leq p_c$, crossover is performed; otherwise, the two parents are copied unchanged into the next generation. A high crossover rate encourages rapid spread of useful genes across the population.

Why Crossover Matters

Crossover is the primary mechanism by which a GA shares and spreads good genetic material across the population. Without it, each chromosome would evolve independently through mutation alone — a slow and inefficient process. Crossover allows a beneficial gene discovered in one chromosome to quickly appear in others, combining partial solutions from different parents into a single, potentially superior offspring. Together, crossover and mutation are complementary: crossover exploits existing good genes by recombining them; mutation explores new regions of the search space by creating genes that were never there before.

3.4.2 Other Crossover Operators

- **Two-Point Crossover:** Two cut points are chosen at random. The segment between them is swapped between the two parents. This allows the middle portion of a chromosome to be exchanged without disturbing the genes at either end, producing a different mixing pattern from single-point crossover.
- **Uniform Crossover:** For each bit position, a fair coin toss decides which parent contributes that bit to the offspring. This produces a more thorough mixing of genes than point-based crossover, and does not favour any particular region of the chromosome.

3.5 Mutation

Crossover can only recombine genes already present in the population. If a useful gene is absent from every chromosome, crossover can never introduce it. Mutation randomly alters individual genes, ensuring the GA can always reach any point in the search space.

3.5.1 Bit-Flip Mutation

In bit-flip mutation, each bit in a chromosome is independently flipped ($0 \rightarrow 1$ or $1 \rightarrow 0$) with probability p_m . For a chromosome of length L , on average $p_m \times L$ bits are flipped per chromosome.

Example 3.8. The population entering the mutation step is:

$$\{1000, 1111, 1011, 1100\}.$$

We set $p_m = 0.1$. For each bit in each chromosome, we draw $r \sim \text{Uniform}[0, 1]$; the bit is flipped only if $r \leq p_m$.

Chromosome 1: 1000 — suppose $r = 0.03 \leq 0.1$ on bit 3; all other bits have $r > 0.1$.

$$\begin{array}{ccc} 1000 & \xrightarrow{\text{flip bit 3}} & 1010 \end{array}$$

Chromosomes 2, 3, 4: 1111, 1011, 1100 — all bits draw $r > 0.1$; no flip occurs. Chromosomes are copied unchanged.

Decoding the full population after mutation ($x \in [0, 10]$, $2^4 - 1 = 15$):

Chromosome	Mutated?	Integer	Phenotype x	Fitness x^2
1010	Yes (bit 3)	10	6.67	44.49
1111	No	15	10.00	100.00
1011	No	11	7.33	53.73
1100	No	12	8.00	64.00

The final population after mutation is {1010, 1111, 1011, 1100}, with average fitness

$$\bar{F} = \frac{44.49 + 100.00 + 53.73 + 64.00}{4} = 65.56.$$

Comparing across all three steps:

Stage	Average fitness	Best fitness
Initial population	33.54	64.00
After selection	59.36	64.00
After crossover	61.54	100.00
After mutation	65.56	100.00

The single bit flip on 1000 raised its fitness from 28.44 to 44.49, pulling the population average up further. The best solution 1111 ($f = 100$) found during crossover is preserved unchanged.

Note on Mutation Rate: The mutation rate p_m is kept deliberately small, typically in the range [0.001, 0.05]. A rate that is too high disrupts good solutions faster than selection can preserve them, reducing the search to a random walk. A rate that is too low allows the population to lose diversity and stagnate. A common rule of thumb is $p_m = 1/L$, which on average flips exactly one bit per chromosome per generation.

Why Mutation Matters

Mutation is the only operator capable of introducing entirely new genetic material into the population. Without it, the GA is confined to combinations of whatever genes were present at initialisation and cannot escape local optima. Together, crossover and mutation are complementary: crossover exploits existing good genes by recombining them; mutation explores new regions of the search space by creating genes that were never there before.

3.5.2 Other Mutation Operators

Bit-flip mutation is the most common operator for binary chromosomes, but other mutation operators exist for different encoding types.

- **Swap Mutation:** Two genes in the chromosome are selected at random and their positions are swapped. This is commonly used with permutation encoding (e.g. TSP), where flipping a bit would produce an invalid route.

$$1 \ 0 \ 1 \ 1 \xrightarrow{\text{swap positions 2 and 4}} 1 \ 1 \ 1 \ 0$$

- **Scramble Mutation:** A random subset of genes is selected and their order is shuffled randomly. This introduces more variety than a single swap and is also commonly used with permutation encoding.

- **Inversion Mutation:** A random segment of the chromosome is selected and the order of genes within that segment is reversed. For example:

$$1 \ 0 \ 1 \ 1 \ 0 \xrightarrow{\text{invert positions 2-4}} 1 \ 1 \ 1 \ 0 \ 0$$

- **Gaussian Mutation:** Used with real-coded encoding. A small random value drawn from a Gaussian (normal) distribution is added to a gene. This produces a fine-grained perturbation around the current value, rather than a complete random replacement.

For the binary encoding used throughout these notes, bit-flip mutation is the standard choice. The operators above become relevant when the encoding changes — swap and scramble for permutation problems, Gaussian for real-coded problems.

3.6 A Complete GA Run

The table below summarises how the population evolved across the three operator steps of Generation 1, using the running example ($f(x) = x^2$, $x \in [0, 10]$, $L = 4$, $N = 4$).

Stage	Population	Average fitness	Best fitness
Initial	{1011, 0110, 1100, 0001}	33.54	64.00
After selection	{1011, 1100, 1011, 1100}	59.36	64.00
After crossover	{1000, 1111, 1011, 1100}	61.54	100.00
After mutation	{1010, 1111, 1011, 1100}	65.56	100.00

Note: The average fitness after selection (59.36) reflects the mating pool {1011, 1100, 1011, 1100}, which has a higher average than the initial population because weaker chromosomes were not selected. Selection does not create new chromosomes — it only decides which ones are allowed to reproduce.

In a single generation, the average fitness nearly doubled and the global optimum $f = 100$ was reached. Selection concentrated the stronger chromosomes; crossover combined their best bits to produce 1111; mutation refined a weaker offspring. Each operator played a distinct and complementary role.

3.6.1 Elitism

Elitism is not part of the basic GA loop described in this article, but it is commonly added in practice.

Elitism: The best chromosome (or best few) from the current generation is copied directly into the next generation, bypassing selection, crossover, and mutation entirely. This guarantees that the best solution found so far is never accidentally lost due to the stochastic nature of the operators.

Elitism is simple to implement and often improves convergence speed in practice. The risk, however, is that copying the same elite chromosome repeatedly can reduce population diversity and contribute to premature convergence if the mutation rate is too low.

3.7 Parameters and Practical Guidelines

3.7.1 The Main Parameters

A GA has five primary parameters.

- **Population size N :** A larger population explores more of the search space but increases computational cost per generation. Typical values range from 20 to 200.
- **Number of generations T :** Controls how long the GA runs. Each generation applies selection, crossover, and mutation once. A larger T allows more refinement but increases total computation. Typical values range from 100 to 1000, depending on problem complexity. The total number of fitness evaluations is approximately $N \times T$.
- **Crossover rate p_c :** Controls how often crossover is applied to each pair of parents. A high value (0.6–0.9) encourages rapid mixing of genetic material.
- **Mutation rate p_m :** Controls how often individual bits are flipped. A low value (0.001–0.05) introduces diversity without disrupting good solutions. A common rule of thumb is $p_m = 1/L$.
- **Chromosome length L :** Determined by the required precision ε via

$$2^L - 1 \geq \frac{x_{\max} - x_{\min}}{\varepsilon}.$$

A summary of typical parameter ranges is given below.

Parameter	Role	Typical range
N	Population size	20 – 200
T	Number of generations	100 – 1000
p_c	Crossover rate	0.6 – 0.9
p_m	Mutation rate	0.001 – 0.05
L	Chromosome length	from precision ε

3.7.2 Premature Convergence

Premature convergence occurs when the population becomes nearly identical before a satisfactory solution has been found. Once all chromosomes are the same, crossover produces no new solutions and the search stalls. Remedies include increasing p_m , using tournament or rank selection to moderate pressure, applying elitism, or restarting with a fresh sub-population.

Summary

- **Encoding:** binary strings; decode via integer then scale to $[x_{\min}, x_{\max}]$.
- **Fitness:** $F = f$ for maximisation; $F = 1/(1 + f)$ or $F = C_{\max} - f$ for minimisation.
- **Selection:** roulette wheel, tournament, or rank; elitism protects the best solution.
- **Crossover:** single-point, two-point, or uniform; rate p_c is high.
- **Mutation:** bit-flip; rate p_m is low; the only source of new genetic material.
- **Parameters:** N , T , p_c (high), p_m (low), L (from precision). Avoid premature convergence via diversity-preserving strategies.

4. Advantages and Limitations

4.1 Advantages

Genetic Algorithms offer several properties that make them attractive for a wide class of optimization problems.

- **No gradient required:** A GA needs only a fitness function — no derivatives, no continuity assumptions, no analytical formula. This makes it applicable to problems where classical methods cannot even begin.
- **Broad applicability:** The same algorithm applies to discrete, continuous, multimodal, black-box, and NP-hard problems with minimal modification. Only the encoding and fitness function change; the core loop stays the same.
- **Global search:** By maintaining a diverse population and combining exploration (mutation) with exploitation (selection and crossover), a GA is less likely to get permanently trapped in a local optimum than a gradient-based method that follows a single trajectory.
- **Parallelism:** Because the GA evaluates an entire population simultaneously, fitness evaluations can be distributed across multiple processors, making it well suited to modern parallel computing environments.

4.2 Limitations

A GA is not without drawbacks, and understanding them is essential for applying it responsibly.

- **No optimality guarantee.** A GA is a heuristic. It returns a good solution, often a very good one, but it cannot certify that the global optimum has been found. For problems where a proven optimal solution is required, exact methods are more appropriate.
- **Parameter sensitivity.** The performance of a GA depends on the choice of N , T , p_c , p_m , and the encoding. Poor choices can lead to premature convergence or unnecessarily slow search. Tuning these parameters typically requires experimentation.
- **Computational cost.** Each generation requires evaluating the fitness of N chromosomes, so the total cost scales with $N \times T$. For problems where a single fitness evaluation is expensive — such as a physical simulation or a real experiment — the total cost can become prohibitive.

5. Conclusion

A Genetic Algorithm distils one of nature’s most powerful ideas into a surprisingly simple computational loop: maintain a population of candidate solutions, score them, favour the better ones, recombine and perturb them, and repeat. No gradient. No closed-form formula. Just selection pressure and variation, applied generation after generation.

What makes this remarkable is not the sophistication of any individual step — each operator is elementary — but the emergent power of running them together. Problems that are discrete, multimodal, black-box, or combinatorially explosive, problems that defeat classical methods outright, often yield to this simple loop.

The GA is not a universal solver. It makes no guarantees, and it requires care in design and tuning. But as a flexible, gradient-free framework for searching complex spaces, it remains one of the most broadly useful tools in computational optimization — and a testament to how much can be achieved by learning from nature.

Python Implementation

A step-by-step Python implementation of the Genetic Algorithm, using the same running example ($f(x) = x^2$, $x \in [0, 10]$, $L = 4$, $N = 4$) developed in these notes, is available as a Google Colab notebook on GitHub.

github.com/kanchan999

Note: After completing these notes, proceed to the companion Question Bank for self-assessment.

Best wishes.
